

Wheel of Fortune (wheel)

William is playing the Wheel of Fortune, a famous game show. The wheel is a “sliced” circle, with numbers on each slice. It can be seen as a circular array of $2N$ elements.

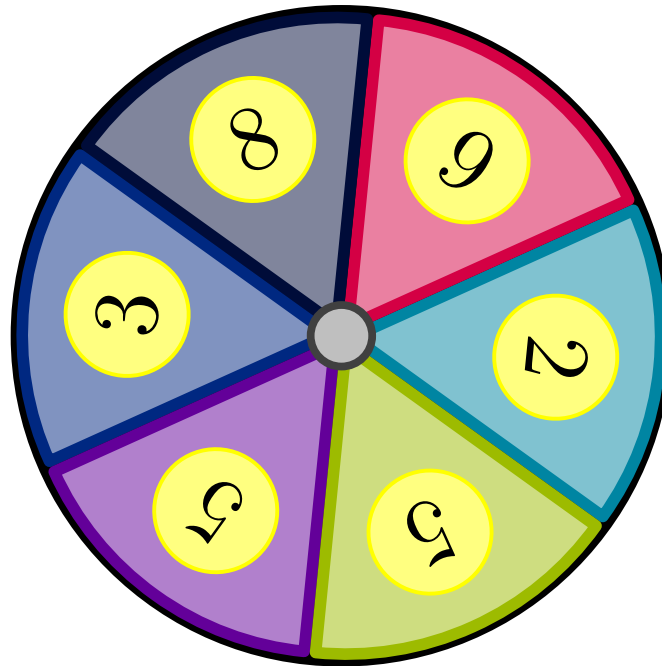


Figure 1: A valid wheel having $N = 3$.

The rules are as follows. William can “turn” the wheel by rotating the array, i.e. moving each of its elements ahead by some positions (possibly making several complete turns). For example, if the array is 2 1 5 3, a rotation of 1 would yield the following array: 3 2 1 5. Rotating again by 2 would yield 1 5 3 2. Rotating again by 1 would yield the original array.

In order to win the game, William needs some information about the wheel everytime he turns it. Specifically: he needs to know the **min** among the first N elements and the **max** among the last N elements. Initially, in the previous example, the min and max values are 1 and 5. After the first rotation they become 2 and 5, then they become 1 and 3, and finally they become 1 and 5 again when the original ordering is restored.

Note: the “turns” add up, that is, the wheel is *not* reset after each turn.

 Among the attachments of this task you may find a template file `wheel.*` with a sample incomplete implementation.

Input

The first line contains a single integer N : half of the length of the wheel. The second line contains $2N$ integers W_i . The third line contains a single integer Q : the number of turns. The fourth line contains Q space-separated integers K_i , the list of “turns” William will do.



Output

You need to write Q lines, each of them corresponding to a turn and containing two integers:

- The **min** of the first half of the wheel after performing the turn.
- The **max** of the second half of the wheel after performing the turn.

Constraints

- $1 \leq N, Q \leq 40\,000$.
- $1 \leq W_i \leq 40\,000$ for each $i = 0 \dots N - 1$.
- $1 \leq K_i \leq 40\,000$ for each $i = 0 \dots Q - 1$.

Scoring

Your program will be tested against several test cases grouped in subtasks. In order to obtain the score of a subtask, your program needs to correctly solve all of its test cases.

- **Subtask 1** [5 points]: Examples.
- **Subtask 2** [25 points]: $N, Q \leq 10$.
- **Subtask 3** [40 points]: $N, Q \leq 100$.
- **Subtask 4** [30 points]: No additional limitations.

Examples

input.txt	output.txt
2 2 1 5 3 3 1 2 1	2 5 1 3 1 5
3 2 9 8 3 5 5 3 2 1 8	2 9 3 9 3 5

Explanation

The **first sample case** is the same example presented earlier.

In the **second sample case** the wheel 2 9 8 3 5 5 gets turned by 2 and becomes 5 5 2 9 8 3, then 1 and becomes 3 5 5 2 9 8, then 8 and becomes 9 8 3 5 5 2.